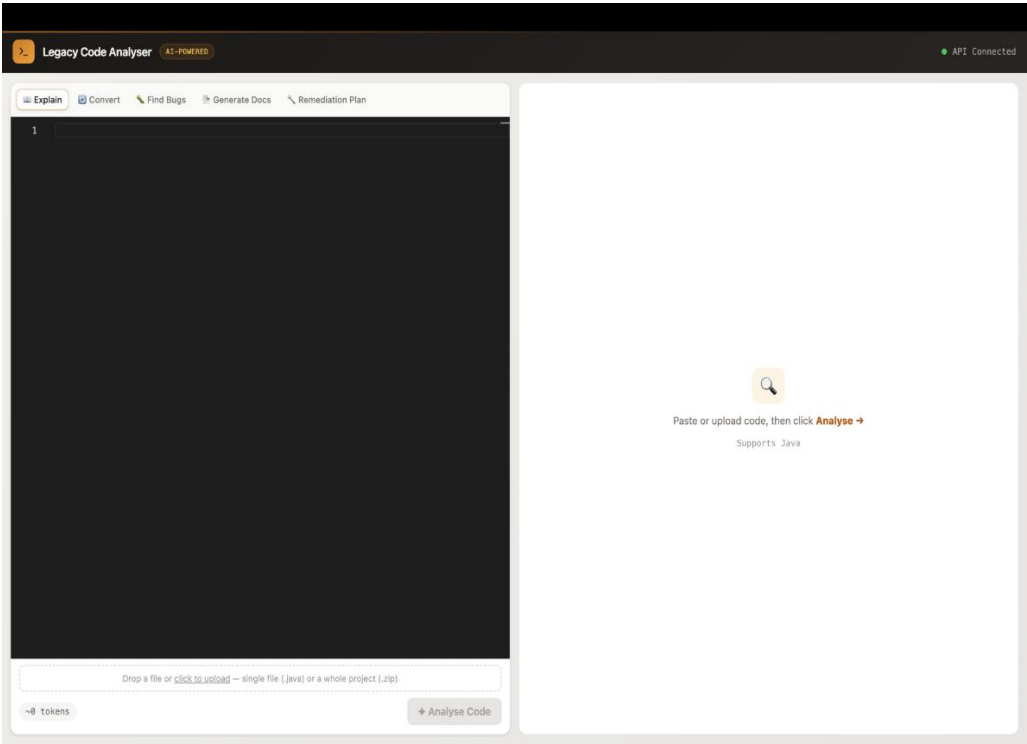
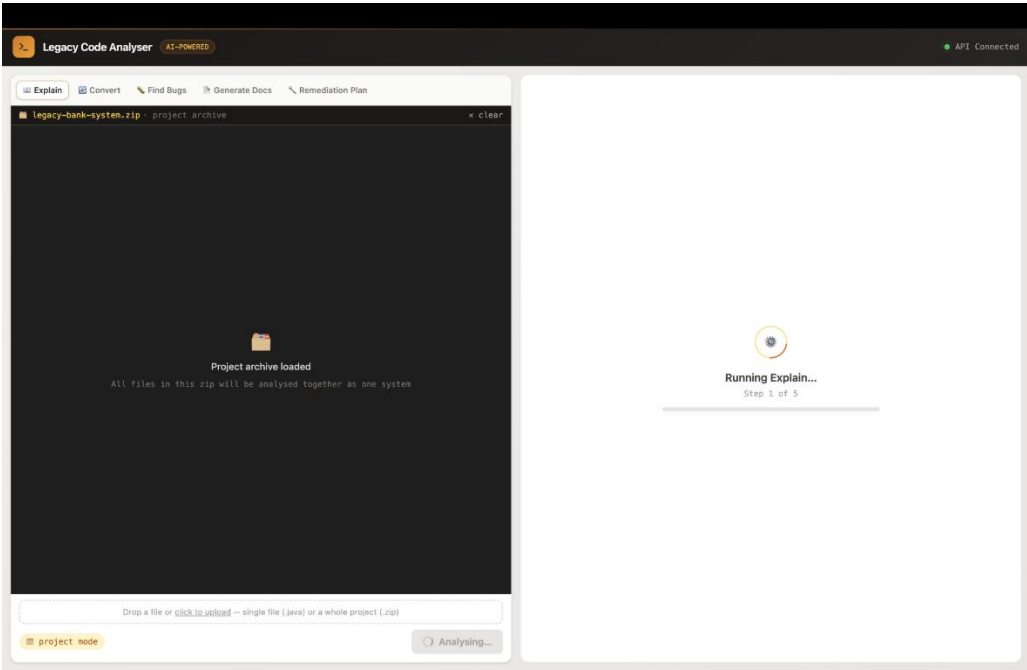


Legacy Code Analyser

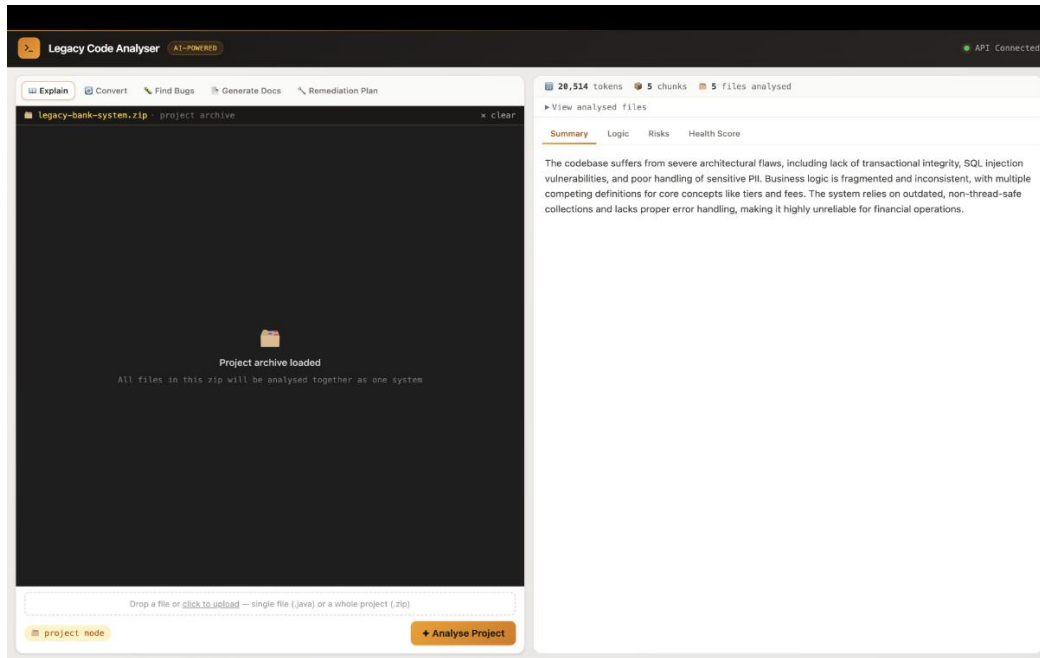
Home Screen interface:



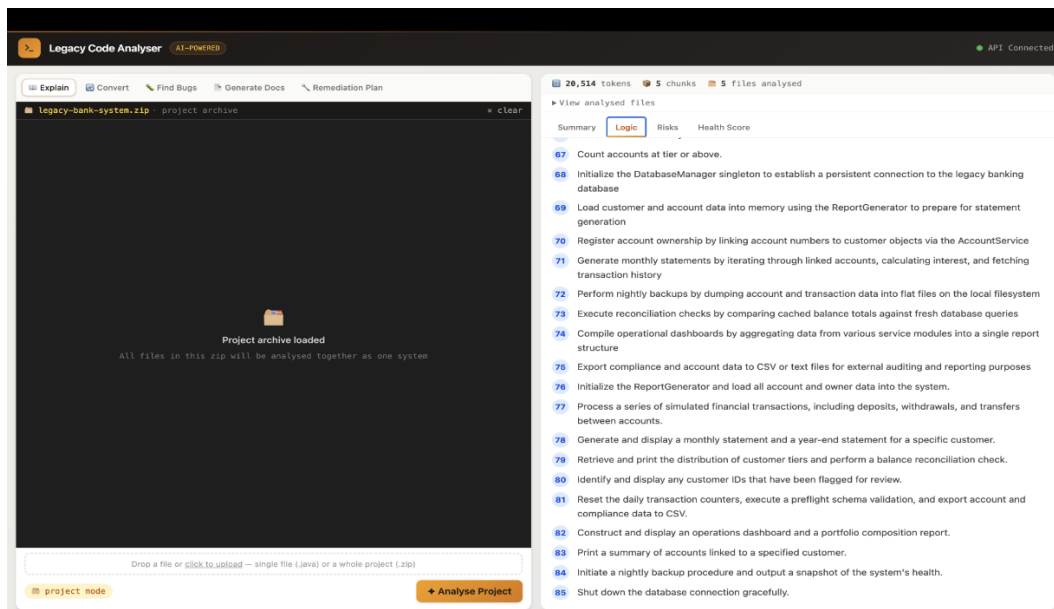
A legacy system with several files has been uploaded and analysis is started



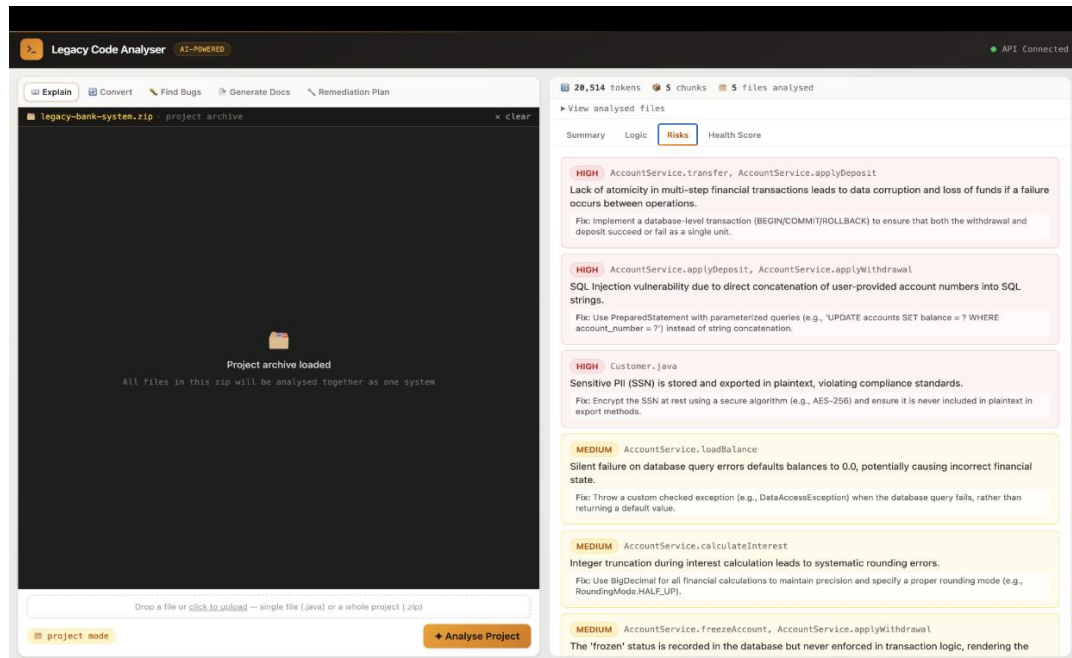
Summary Section: A small summary section is generated



Code logic: The legacy system uploaded ~3000 lines and it gives us 80+ line of breakdown , step by step which helps us understand the system easily



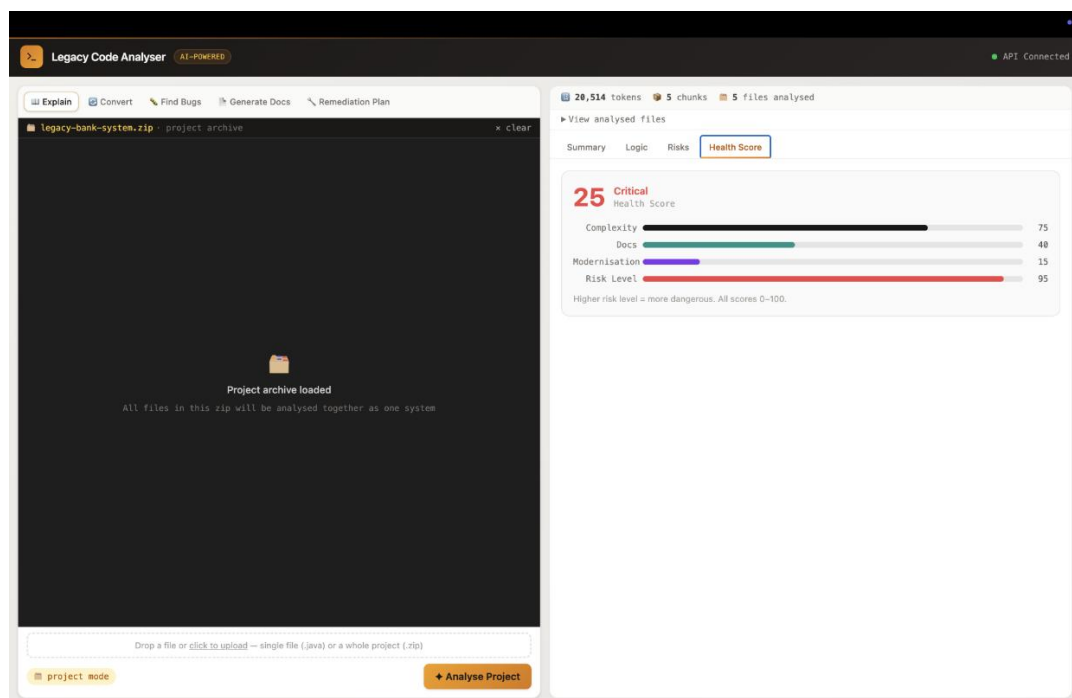
Risks: give us the list of the risk of three different level (high , medium , low) on the basis of how it is impacting the system



The screenshot shows the Legacy Code Analyser interface with the 'Risks' tab selected. The interface displays a list of risks categorized by severity: HIGH, MEDIUM, and LOW. The risks are related to account services, SQL injection, and sensitive data handling.

- HIGH** AccountService.transfer, AccountService.applyDeposit
Lack of atomicity in multi-step financial transactions leads to data corruption and loss of funds if a failure occurs between operations.
Fix: Implement a database-level transaction (BEGIN/COMMIT/ROLLBACK) to ensure that both the withdrawal and deposit succeed or fail as a single unit.
- HIGH** AccountService.applyDeposit, AccountService.applyWithdrawal
SQL Injection vulnerability due to direct concatenation of user-provided account numbers into SQL strings.
Fix: Use PreparedStatement with parameterized queries (e.g., 'UPDATE accounts SET balance = ? WHERE account_number = ?') instead of string concatenation.
- HIGH** Customer.java
Sensitive PII (SSN) is stored and exported in plaintext, violating compliance standards.
Fix: Encrypt the SSN at rest using a secure algorithm (e.g., AES-256) and ensure it is never included in plaintext in export methods.
- MEDIUM** AccountService.loadBalance
Silent failure on database query errors defaults balances to 0.0, potentially causing incorrect financial state.
Fix: Throw a custom checked exception (e.g., DataAccessException) when the database query fails, rather than returning a default value.
- MEDIUM** AccountService.calculateInterest
Integer truncation during interest calculation leads to systematic rounding errors.
Fix: Use BigDecimal for all financial calculations to maintain precision and specify a proper rounding mode (e.g., RoundingMode.HALF_UP).
- MEDIUM** AccountService.freezeAccount, AccountService.applyWithdrawal
The 'frozen' status is recorded in the database but never enforced in transaction logic, rendering the

Health score: It helps us to understand the overall health of the system in 4 regards ie: complexity, documentation, modernization and risk level



The screenshot shows the Legacy Code Analyser interface with the 'Health Score' tab selected. The interface displays a health score of 25, which is labeled as 'Critical'. The score is broken down into four categories: Complexity (75), Docs (48), Modernisation (15), and Risk Level (95). A bar chart visualizes these scores.

Category	Score
Complexity	75
Docs	48
Modernisation	15
Risk Level	95

Higher risk level = more dangerous. All scores 0-100.

Convert: Modernized Code Following Current Standards for all the files in the system.

The screenshot displays the Legacy Code Analyser interface, which is AI-POWERED and has an API Connected status. The main window is divided into two panels. The left panel, titled 'Project archive', shows a file named 'legacy-bank-system.zip' and a message: 'Project archive loaded. All files in this zip will be analysed together as one system.' Below this, there is a button to 'Analyse Project' and a note to 'Drop a file or click to upload - single file (.java) or a whole project (.zip)'. The right panel, titled 'View analysed files', shows a list of 5 files converted. The first file, 'src/com/legacybank/core/AccountService.java', is selected, and its modernized code is displayed. The code is in Python and includes imports for 'decimal', 'typing', and 'enum'. It defines a 'DatabaseManager' class with methods for 'new_instance', 'get_instance', and 'execute_query'. The second file, 'src/com/legacybank/core/Customer.java', is also shown, with imports for 'dataclasses', 'datetime', 'typing', and 'enum'. The third file, 'src/com/legacybank/core/ReportGenerator.java', is shown with imports for 'datetime', 'typing', and 'pathlib'. The fourth file, 'src/com/legacybank/core/TransactionProcessor.java', is shown with imports for 'java.time', 'java.util.concurrent', and 'java.util'. Each code block has a 'Copy' button next to it.

```
import decimal
from decimal import Decimal, ROUND_HALF_UP
from typing import Dict, List, Optional, Any, Final

# --- Placeholder/Mock Classes for Dependencies ---
# In a real migration, these would be properly converted or replaced with modern
# equivalents.

class DatabaseManager:
    """Mock DatabaseManager for demonstration purposes."""
    _instance: Optional['DatabaseManager'] = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            return cls._instance

    @classmethod
    def get_instance(cls) -> 'DatabaseManager':
        return cls()

    def execute_query(self, table: str, column: str, values: str) -> List[Dict[str, Any]]:
        """Simulates a database query. Returns a list of dicts."""
        print(f"DB: Executing query on {table} where {column} = '{values}'")
        # Simulate some data for testing
        if table == "accounts" and column == "account_number":
            if value == "ACC123":
                return [{"account_number": "ACC123", "balance": "1000.00", "frozen": "0"}]
            if value == "ACC456":
                return [{"account_number": "ACC456", "balance": "-50.00", "frozen": "1"}]
            if value == "ACC789":
                return [{"account_number": "ACC789", "balance": "50000.00", "frozen": "0"}]
        return []
```

```
from dataclasses import dataclass, field
from datetime import date, datetime
from typing import List, Dict, Optional
from enum import IntEnum
```

```
import datetime
from typing import List, Dict, Any, Optional
from pathlib import Path

class ReportGenerator:
```

```
package com.legacybank.core;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
```

Warnings: Identifies critical vulnerabilities, architectural weaknesses, code quality issues, and functional limitations discovered during system-wide analysis.

Legacy Code Analyser

AI-POWERED

API Connected

Explain

Convert

Find Bugs

Generate Docs

Remediation Plan

legacy-bank-system.zip

project archive

clear

Project archive loaded

All files in this zip will be analysed together as one system

Drop a file or [click to upload](#) — single file (.java) or a whole project (.zip)

project mode

Analyse Project

20,514 tokens

5 chunks

5 files analysed

View analysed files

Modern Code

Changes

Warnings

SQL Injection Vulnerability:

The 'executeUpdate' method in 'DatabaseManager' (and its usage in 'AccountService' for 'UPDATE' statements) is highly susceptible to SQL injection. In a production environment, all database interactions must use parameterized queries.

Lack of Transactional Guarantees:

The original code's critical flaw regarding non-atomic operations (e.g., 'apply_deposit's cache vs. DB update, 'transfer's withdrawal then deposit) persists. While basic rollbacks are added for cache consistency, a robust solution requires proper database transactions or an ORM with transaction management.

Unenforced Account Freezing:

The 'freeze_account' and 'unfreeze_account' methods only update a flag in the database. The core transaction methods ('apply_deposit', 'apply_withdrawal', 'transfer') do not check this flag, meaning frozen accounts can still be transacted upon. This requires explicit checks in all relevant transaction methods.

Scheduled Payments Not Executed:

The 'schedule_payment' method merely records payment intent in memory. There is no mechanism or external scheduler implemented to actually process these payments. This feature is non-functional as-is.

Single Account Owner Limitation:

The 'register_owner' and 'add_joint_owner' methods overwrite the 'Customer' object for an account, meaning true joint accounts (with multiple owners) are not supported by the current data model. This requires a change to a one-to-many relationship for account ownership.

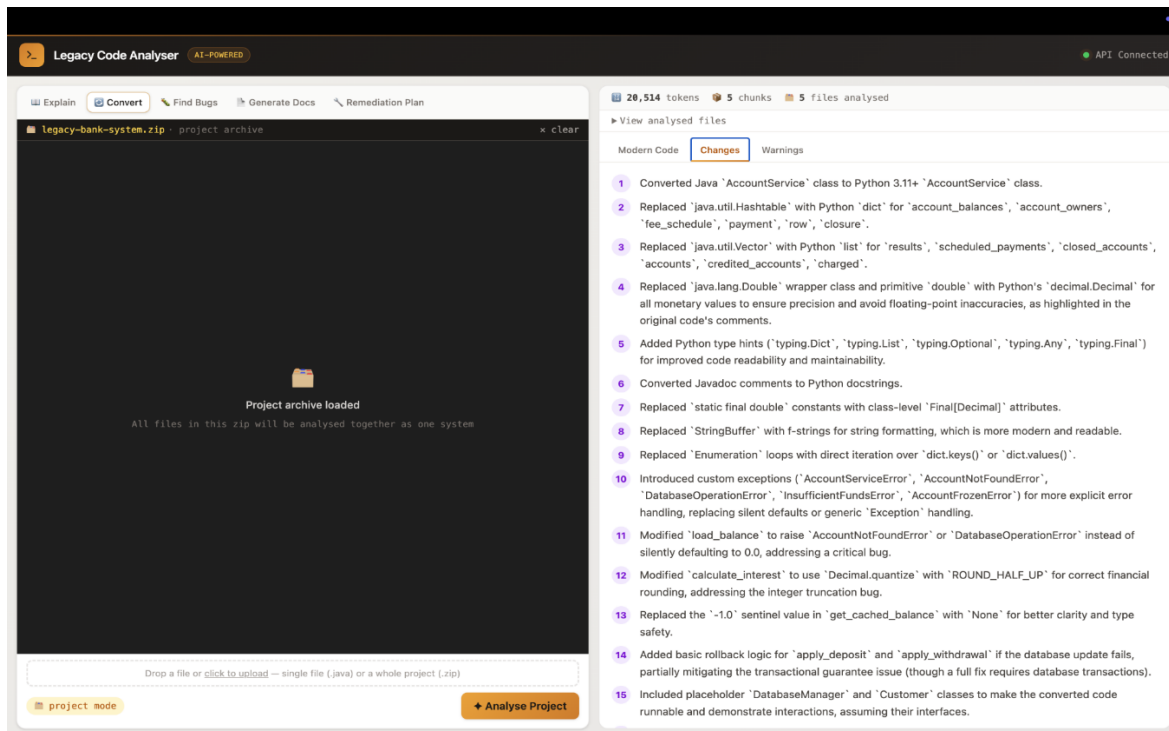
Inconsistent Fee Management:

The 'OVERDRAFT_FEE' is a hardcoded constant, while other fees are managed via 'set_fee'/'get_fee'. 'apply_withdrawal' does not consult the 'fee_schedule'. This creates two disconnected sources of truth for fees, leading to potential inconsistencies. A unified fee management system is needed.

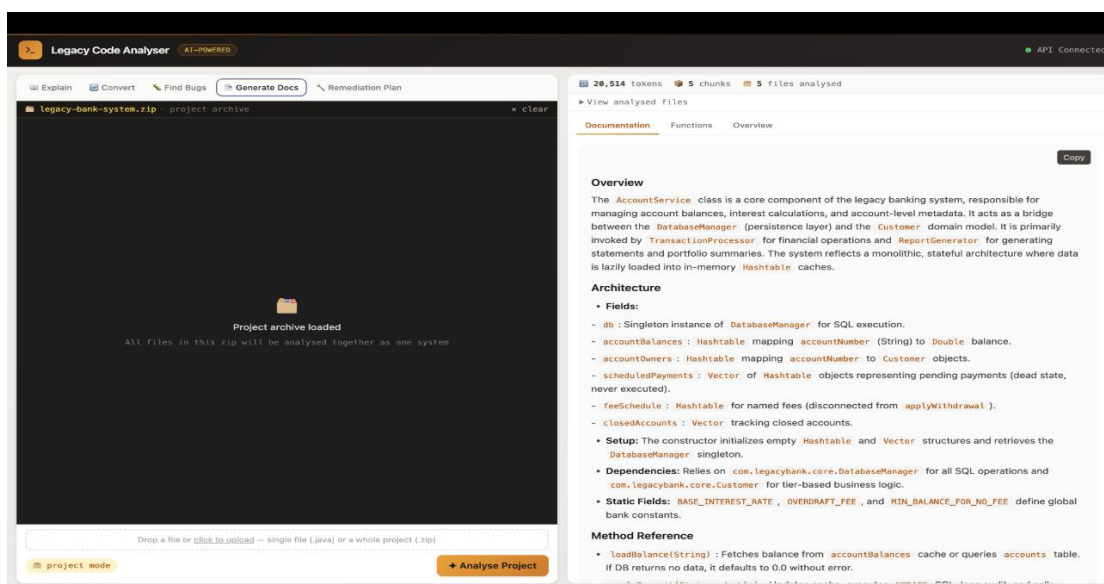
Misleading Aggregate Reports:

'get_sum_of_cached_balances' and 'get_highest_cached_balance' only reflect accounts loaded into the in-memory cache during the current session, not the bank's true overall aggregates. These methods should query the database for accurate, comprehensive results.

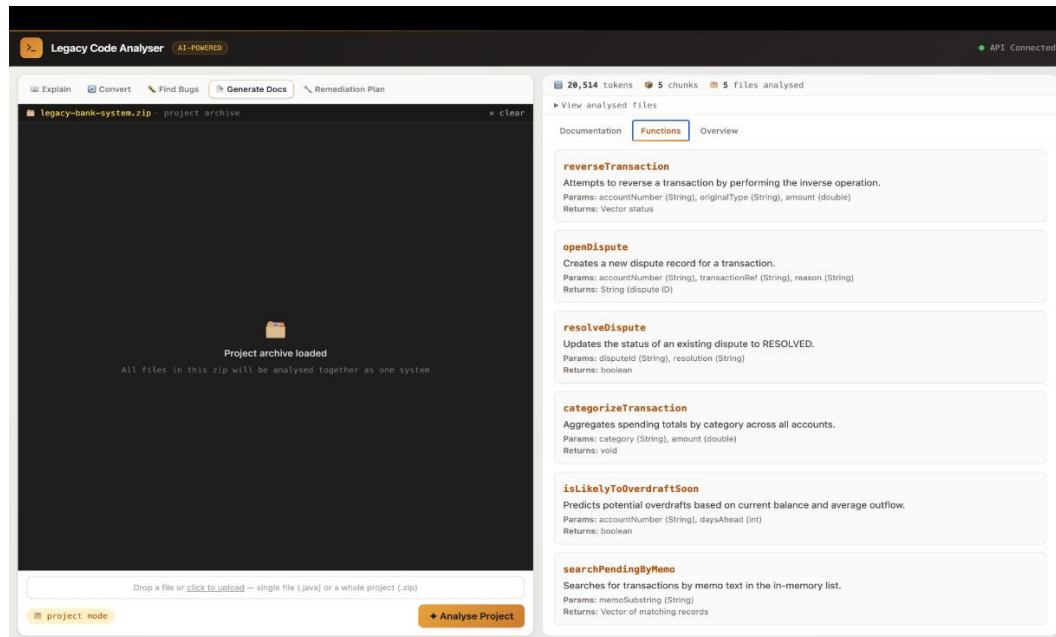
Changes: Provides a detailed analysis of refactoring, architectural improvements, API upgrades, and coding standard adaptations introduced during the legacy-to-modern Java migration process



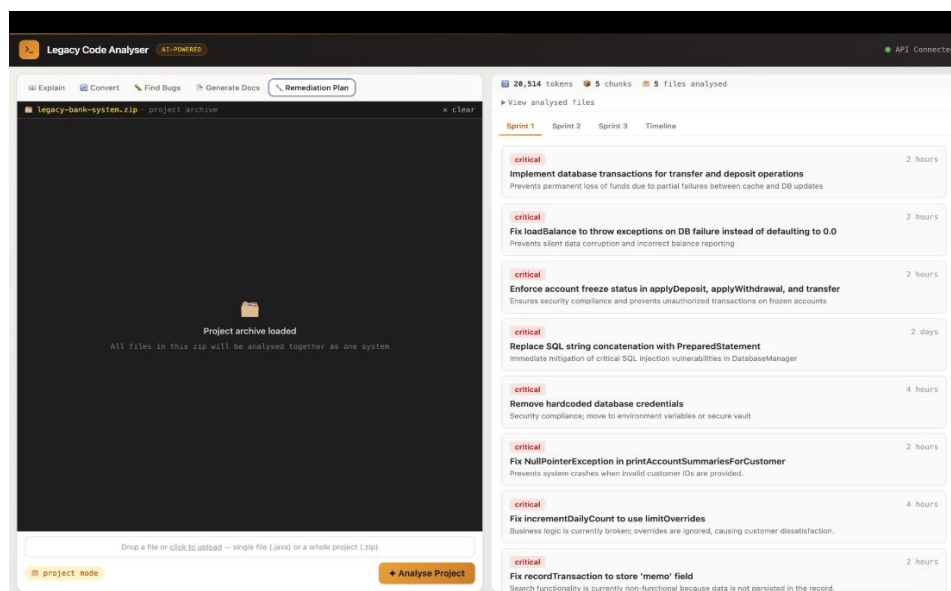
Generate Docs: Develop comprehensive documentation describing the system architecture, component relationships, method traceability, execution flow, data lifecycle, and architectural constraints.



Functions: list of all the function name with brief function description , arguments passed to it and return type



Remediation plan : it categorizes the problems with the legacy code in four section , critical , high ,medium and low and gives provides us the information on what are the issue with the system so that we could plan our fixes according to the priority level and gives us the estimate on how much time it would take to prevent those risks



Legacy Code Analyser

AI-POWERED

API Connected

ExplainConvertFind BugsGenerate DocsRemediation Plan

legacy-bank-system.zip · project archive

Project archive loaded

All files in this zip will be analysed together as one system

Drop a file or click to upload — single file (.java) or a whole project (.zip)

project mode

Analyse Project

28,514 tokens · 5 chunks · 5 files analysed

View analysed files

Sprint 1Sprint 2Sprint 3Timeline

high

Replace Hashtable/Vector with modern Java Collections (HashMap/ArrayList)

Improves type safety, performance, and removes legacy synchronization overhead

1 day

high

Centralize business rules for fees and interest rates

Eliminates conflicting sources of truth and logic drift between AccountService and Customer

1 day

high

Implement proper rounding for interest calculations using BigDecimal

Eliminates systematic rounding errors and ensures financial accuracy

1 day

high

Implement proper JDBC Connection Pooling

Eliminate single-connection bottleneck and prevent resource exhaustion

2 days

high

Add input validation and path sanitization for file operations

Prevent directory traversal attacks in backup and export methods

1 day

high

Implement cache eviction policy for queryCache

Prevent unbounded memory growth and potential OOM errors

1 day

high

Implement memory leak remediation for pendingTransactions

The list grows indefinitely, leading to eventual OutOfMemoryError.

1 day

high

Consolidate FEE_WAIVER_THRESHOLD logic

Prevents business rule drift between TransactionProcessor and AccountService.

1 day

Legacy Code Analyser

AI-POWERED

API Connected

ExplainConvertFind BugsGenerate DocsRemediation Plan

legacy-bank-system.zip · project archive

Project archive loaded

All files in this zip will be analysed together as one system

Drop a file or click to upload — single file (.java) or a whole project (.zip)

project mode

Analyse Project

28,514 tokens · 5 chunks · 5 files analysed

View analysed files

Sprint 1Sprint 2Sprint 3Timeline

medium

Refactor Customer SSN storage to use encryption at rest

Addresses critical compliance and security vulnerability

3 days

medium

Implement proper account ownership model (One-to-Many)

Fixes the joint account bug where owners are silently overwritten

3 days

medium

Replace manual SQL string concatenation with PreparedStatement

Prevents SQL injection vulnerabilities and improves query performance

3 days

medium

Refactor ReportGenerator to use proper logging and monitoring

Replace stdout/stderr with structured logging for observability

3 days

medium

Introduce Transaction Management (ACID)

Ensure data integrity during multi-step operations and batch processing

5 days

medium

Migrate from legacy Vector/Hashtable to Java Collections Framework

Improve performance, type safety, and code maintainability

4 days

medium

Refactor Hashtable/Vector to Java Collections Framework

Modernizes type safety and performance, replacing legacy thread-safe overhead.

3 days

medium

Implement proper Dispute and Transaction domain objects

Replaces loose HashTables with strongly-typed classes to prevent runtime key-missing errors.

3 days

Legacy Code Analyser

AI-POWERED

API Connected

Explain

Convert

Find Bugs

Generate Docs

Remediation Plan

legacy-bank-system.zip

project archive

clear

Project archive loaded

All files in this zip will be analysed together as one system

Drop a file or [click to upload](#) — single file (.java) or a whole project (.zip)

project mode

Analyse Project

29,514 tokens

5 chunks

5 files analysed

View analysed files

Sprint 1

Sprint 2

Sprint 3

Timeline

Total Modernisation Estimate

12 days of focused engineering effort to stabilize, secure, and modernise the core banking logic.

Sprint 1 — Critical fixes (8 tasks)

Sprint 2 — High priority improvements (8 tasks)

Sprint 3 — Modernisation (9 tasks)

X`